



COMPETITION PACK

Cloud Computing Day 1

2026 GLOBAL SKILLS CHALLENGE

WOLLONGONG, AUSTRALIA
24 - 28 June 2026





CONTENTS

CONTENTS	2
PROJECT INFORMATION	3

TASK DESCRIPTION

Astra Exchange is the Star Bourse, a trading floor where the great houses of the settled solar systems buy and sell stakes in newly catalogued stars. When a survey ship confirms a new sun/star, a stake in it opens for trade (a claim); VEGA, NOVA, ORION, LYRA, ATLAS, HELIO, RIGEL, MIRA, and the hundreds of fainter lights behind them.

Brokers across thirty systems submit claims continuously, and the Astra Exchange's promise to every house is the same one it has made for nine hundred years: your claim is confirmed on the fastest open floor, and the fastest floor is chosen for you.

The confirmation work happens on a fleet of matching engines called the "floors". Each floor is identical hardware running the identical logic, but at any given moment they are not equally fast. One floor could be in the mid-reconciliation and sluggish; the one beside it would have cleared its book and is quick. A floor advertises, second by second, how long it will take to confirm the next claim handed to it. The platform's entire job is to read those advertisements, decide which floor is quickest *right now*, and hand the claim there, so that the broker's confirmation comes back before the next two-minute trading window closes.

The platform you have inherited does not do this yet. The previous engineering team wired the claim intake to the floors with a plain round-robin: every claim goes to the next floor in line, fast or slow, blind. It worked when there were two floors and the book was quiet. Last quarter, during a supernova rush, claims were landing on floors that were three seconds deep while idle floors sat next to them doing nothing. Confirmations missed their windows.

The House of Rigel's standing order to buy was confirmed only after forty seconds; late at a price that had already moved, and Rigel's counsel sent a letter Astra Exchange's board; is a warning to the exchange. Astra has recruited you to complete the architecture.

The team sketched the fix on the whiteboard before they left.

- A poller that discovers every live floor and reads its advertised speed on a tight loop.
- A fast shared ranking of floors by speed.
- A router that, for each incoming claim, picks the quickest floor off that ranking and forwards the claim to it.
- A durable record of every confirmation so the registry of claims can be read back with certainty.

You inherited this platform forty minutes ago. The TitanFlow engine is ready to score you. It started thirty seconds before you sat down, and right now it is waiting to hammer your architecture with a sequence of requests.

CAUTION

- **Duration:** 6 hours
- **Region:** us-east-1
- **IAM:** You have Administrator access and you create the roles this platform needs. Every role you create must follow least privilege scoped to the specific actions and resources the workload uses.

INITIAL STATE

A partially built platform has been provisioned in collaboration with the engineering team. The scaffolding listed under Service Details exists already and must be used as-is. Do not recreate or delete it. The two container registries are empty when the clock starts, so building and pushing both images is the first action; until the images are present, nothing runs and the engine sees only failures.

The platform serves traffic in two routing modes from a single router image. In `round_robin` mode the router forwards each claim to the next floor in line, blind to speed; this gets the platform live and moves the score slightly upward. In `optimized` mode a background poller reads every floor's advertised speed on a four-second loop, ranks the floors in a Redis sorted set, and routes each claim to the quickest floor; this is the state that earns the score. The same image runs both modes, selected by one environment variable.

THREE STAGES

Stage One: Ingestion and Routing

A public edge (API Gateway in front of an `Application Load Balancer`) receives each claim at `POST /submit` and passes it to the router. The `router` discovers the live floors through a headless Service and forwards the claim to a floor. Every confirmation is written to a `DynamoDB` table and echoed back to the caller with the floor's signed audit header intact.

Stage Two: Ranking

A background poller reads each live floor's advertised speed from its `/health` route on a tight loop and maintains a live ranking of floors by speed in an `ElastiCache` (Redis or Valkey) sorted set. Stale floors are dropped from the ranking, so a dead or retired floor stops receiving claims.

Stage Three: Confirmation and Clearing

Each confirmation is durably recorded and is readable back with a strongly consistent read at `GET /trades/{trade_id}`. Separately, a high-value settlement instruction arrives at `POST /clearing/instruction` roughly every ten minutes; it is verified, margin-checked, branched, cleared through an external Clearing Authority, and returned as a signed receipt.

Pre-Provisioned Resources

Resource	Name	Notes
VPC	titanflow-astra-vpc	10.42.0.0/16, public and private subnets across two AZs, EKS and ELB subnet tags applied.
EKS cluster	titanflow-astra-cluster	Kubernetes 1.34, three-node managed group, AWS Load Balancer Controller and metrics-server installed, CloudWatch observability addon installed but not yet permitted to publish, OIDC provider associated.
ECR repositories	astra-worker, astra-router	Empty. Push your images here first.

x water wle
x water wle

Resource	Name	Notes
DynamoDB	titanflow-astra-worker-registry	Floor liveness heartbeats (worker_id, pod_ip, ts_ms) every two seconds. Do not write to it yourself.
Secrets Manager	titanflow-astra-floor-secret	Floor-signing key, read by the floor through IRSA.
Secrets Manager	titanflow-astra-clearing-secret	Static clearing key for settlement verification. Never expose it in an environment variable.
EC2	titanflow-astra-engine	TitanFlow load generator. Running. Do not stop it.

Database

Confirmation table (create): DynamoDB titanflow-astra-trades, on-demand billing, with a TTL attribute named expires_at.

Key name	Key type	Data type
trade_id	Partition Key	String

Registry table (pre-provisioned, do not write to it): DynamoDB titanflow-astra-worker-registry. Non-key heartbeat attributes are pod_ip and ts_ms.

Key name	Key type	Data type
worker_id	Partition Key	String

Container Configuration

astra-worker:1.0.0, the floor. Routes: GET /health and POST /process.

Variable	Value
REGISTRY_TABLE_NAME	titanflow-astra-worker-registry (required)
FLOOR_SECRET_NAME	titanflow-astra-floor-secret (required)
REGISTER_INTERVAL_S	2
WORKER_ID	Leave unset; defaults to the pod name through the Downward API.

Give the floor container a modest CPU request, for example 250m, because the autoscaler measures CPU against it. The advertised-speed band defaults (300 to 3000 ms, re-rolled every 20 s) are correct and need not be set. The floor reads its signing secret through IRSA, so do not pass the secret value as a variable.

astra-router:1.0.0, the poller and router. Routes: POST /submit, GET /trades/{trade_id}, GET /workers.

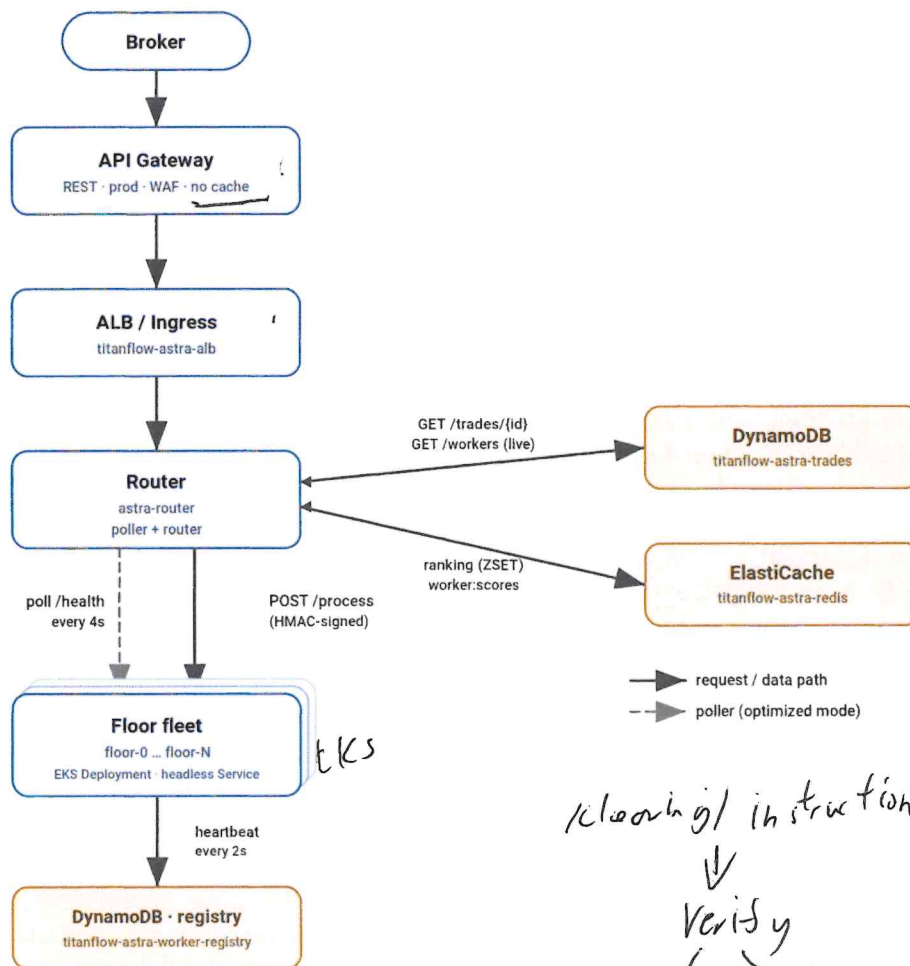
Variable	round_robin	optimized
ROUTING_MODE	round_robin	optimized
WORKERS_SERVICE_DNS	astra-workers-headless	astra-workers-headless
TRADES_TABLE_NAME	titanflow-astra-trades	titanflow-astra-trades
REDIS_HOST	not used	your titanflow-astra-redis primary endpoint
REDIS_PORT	not used	6379
POLL_INTERVAL_S	not used	2
POLL_TIMEOUT_MS	not used	600
STALE_MS	not used	8000
HISTORY_FLUSH_S	not used	30

The optimized router maintains a Redis sorted set named worker:scores, ordered by **advertised speed** in **milliseconds**, so the quickest floor is always the first element. The poller populates it; you do not write to it yourself.

Clearing Workflow

- POST /clearing/instruction starts an **Express state machine** synchronously and returns its output as the HTTP response body.
- Verify: recompute the HMAC with titanflow-astra-clearing-secret over the canonical string claim_id|nonce|house|leg_id:side:amount,...|posted_collateral, with legs comma-joined in payload order.
- Net exposure equals the sum of debit amounts minus the sum of credit amounts.
- Branch (a Choice state): a forged signature goes to quarantine; a valid signature with exposure greater than collateral goes to quarantine, where equality settles; a valid signature within collateral settles.
- Clear externally at <https://api.titanflow.ethnustech.com>: first POST /authority/token for a single-use token that expires in thirty seconds, then POST /gateway/settle or POST /authority/quarantine. All calls require the x-api-key header and are limited to ten requests per second, so use retry with backoff and catch.
- Return the Authority's signed receipt unmodified. On quarantine, also publish a notification to the titanflow-astra-alerts SNS topic.
- Scoring: a correct clearing is worth +10,000; a wrong terminal, an invalid or missing receipt, or a timeout is worth -8,000.

Architecture



clearing / instruction
 ↓
 Verifying
 ↙ ↘
 forged not forged

Task at Hand

Build the platform the previous team sketched and bring it alive from end to end. Package and publish the two provided application images, stand up the fleet of matching floors and the routing tier from them, and place a secured public entry point in front so the scoring engine can reach you. Bring the routing tier up first in its simple pass-through mode to get traffic flowing and the score moving, then switch it into its intelligent mode, in which a background loop continuously discovers every live floor, reads the speed each floor is advertising, keeps a fast shared ranking of them, and forwards every incoming claim to whichever floor is quickest at that instant. Every confirmation must be written to durable storage and be immediately readable back with certainty, and the signed audit stamp the floor returns must reach the caller untouched.

Then make the platform production grade against the qualities this brief calls out. Give every identity you create only the permissions it needs, scoped to the exact resources it touches, with no broad or wildcard grants. Shield the public edge from abusive traffic, keep the platform responsive and self-scaling as load surges and falls, and make it observable through logs, alarms, tracing, and a single view of its health. Keep a durable backup of the confirmation record, hold the whole design lean and cost aware, and keep the account tidy, with consistent naming and tagging and no abandoned resources left behind.

Finally, add a separate high-value settlement path that runs alongside the trading flow. A privileged settlement request arrives periodically and must be authenticated by recomputing its signature, then checked to confirm the committed funds cover its exposure. Based on that outcome, route it down a settle or a quarantine decision, clear it through the external Clearing Authority, and return the authority's signed receipt unchanged. Whenever a request is quarantined, raise an alert so the desk is notified.

Beyond the broad strokes, build the platform's observability and housekeeping out in full. Create a single operational dashboard that brings the whole platform into one view: the public edge's request rate and error rate, the routing tier and floor fleet's compute load, the confirmation store's consumed capacity and any throttling, and the ranking cache's health. Pair it with alarms on the conditions that actually break the platform, namely a rise in edge errors, compute saturation on the floors, throttling on the confirmation store, and a fall in the number of healthy floors, and route every alarm to a notification channel backed by a durable queue so no alert is lost. Turn on end-to-end request tracing at the public entry point, and grant the cluster's pre-installed telemetry agent the permission it needs to publish container logs, since it ships nothing until you allow it. Capture network flow logs for the platform's network, deliver the load balancer's access logs to durable storage, and publish the two images to registries that block silent tag overwrites and scan each image on push. Finally, keep the platform from accumulating cost and clutter as it runs: size every component to its real load, bound the retention on every log group, expire stored access logs and old or untagged images through lifecycle rules, let confirmation records age out on their own through a time-to-live attribute, and let autoscaling release idle capacity when load falls.

Scoring

A live engine scores the platform continuously. Routing a claim to the fastest live floor at that moment scores full; routing to a valid but slower floor scores a small positive; a forged or missing audit signature, stale routing to a dead floor, a timeout, or a stale read subtracts. In round_robin the fastest-floor match happens by luck about one submission in five and the score crawls; once the poller is live and the ranking is fresh, the match rate jumps and the score climbs. At the end of the session a static review checks resource state against this brief, and the final score combines the live engine score and the static review.

Reference Document

- <https://docs.aws.amazon.com/>
- <https://docs.aws.amazon.com/eks/latest/userguide/what-is-eks.html>
- <https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html>
- Clearing Authority base URL: <https://api.titanflow.ethnustech.com>